Step 1 — The Database and the Safety Guard — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the database pieces
- Hint 2 — the guard, and the easy-to-miss parts

Hint 1 — the database pieces

SQLite ships with Python (`import sqlite3`), so there's no server to run. Three small functions:

```
build_sample_db(db_path)   # create customers + orders, insert rows (once)
schema_text(db_path)       # SELECT sql FROM sqlite_master WHERE type='table'
run_select(db_path, sql)   # connection.execute(sql).fetchall()
```

`schema_text` returns the `CREATE TABLE` statements joined into one string — that's what the model reads to know the tables and columns. `build_sample_db` should do nothing if the file already exists, so running `main.py` twice doesn't double the rows.

Hint 2 — the guard, and the easy-to-miss parts

The guard is the safety of the whole project, and it's plain Python:

```
is_safe_select(sql):
    cleaned = sql.strip().rstrip(";").strip()   # allow one trailing ;
    if not cleaned.lower().startswith("select"): return False
    if ";" in cleaned: return False             # no chained statements
    for word in FORBIDDEN_WORDS: ...            # no insert/update/delete/drop
    return True
```

Two easy-to-miss parts: **strip one trailing semicolon** (a model often ends a query with `;`) but then **reject any remaining `;`** so `SELECT ...; DROP TABLE ...` can't sneak through; and keep the forbidden list lowercase and compare against the lowercased query. Test it with a plain `SELECT`

(passes) and with `INSERT` / `UPDATE` / `DELETE` / `DROP` /chained (all rejected). The whole file set is in the Solution.